Introduction

Abdelkader Ouared

abdelkader.ouared@univ-tiaret.dz


@abdelkader-ouared

Exemple de Motivation

Commençons par une situation concrète !

- Inspirons-nous de **Jean Piaget** : apprendre, c’est construire ses connaissances en réfléchissant à des situations-problèmes.
- *Avant de découvrir DevOps, réfléchissons ensemble à un problème réel de développement logiciel?*



Motivation (1/10)

Votre mission

Projet : le portail numérique de l'université

L'université vous demande de développer un portail web avant le début des inscriptions.

3 équipes :

- Équipe 1 : Inscription & étudiants
- Équipe 2 : Emploi du temps & examens
- Équipe 3 : Actualités & notifications

Le portail contient des données personnelles d'étudiants.

Votre mission : livrer une application fonctionnelle, fiable et sécurisée.



Motivation (2/10)

Vos contraintes
Les règles du projet
Vous devez :

- travailler à **3 équipes en parallèle** ;
- intégrer vos trois modules ;
- accepter les changements du client ;
- tester l'application ;
- la déployer sur le serveur universitaire ;
- **protéger les données des étudiants** ;
- livrer avant la date limite.

Vous avez 2 semaines.



Motivation (3/10)

Chacun commence son travail
Les trois équipes développent leurs modules.
« Nous allons d'abord terminer notre module. »
Mais une contrainte importante est annoncée :



- **Les données sont sensibles**
- Le système contient :
  - informations personnelles ;
  - comptes étudiants ;
  - résultats/examens ;
  - données d'inscription.

« Une faille de sécurité dans un seul module peut mettre en danger tout le système. »

Question :
« À quel moment devons-nous nous occuper de la sécurité ? »

Motivation (4/10)

Première difficulté : l'intégration



Les trois modules fonctionnent séparément.
Mais lorsque vous les assemblez :

- ❌ incompatibilités
- ❌ erreurs d'interface
- ❌ problèmes de données
- ❌ régressions

« Ça fonctionne dans mon module ! »
Question :
« Comment éviter de découvrir les problèmes seulement à la fin ? »

Motivation (5/10)

Le client change une fonctionnalité

Le service de scolarité demande :

« Lorsqu'un examen est modifié, l'étudiant doit recevoir automatiquement une notification. »

Cela implique : **Équipe 2 ↔ Équipe 3**

- Une modification dans un module affecte l'autre.



Question :

« Comment pouvons-nous intégrer les changements sans casser ce qui existe déjà ? »

Motivation (6/10)

Les tests

Vous commencez à tester régulièrement.



Vous trouvez :

- ❌ bugs
- ❌ régressions
- ❌ problèmes d'intégration

Mais vous constatez quelque chose :

« Nous testons seulement quand le développement est presque terminé. »

Question :

« Ne serait-il pas préférable de détecter les problèmes plus tôt ? »

Motivation (7/10)

Et la sécurité ?

Le responsable sécurité intervient.

Il demande :

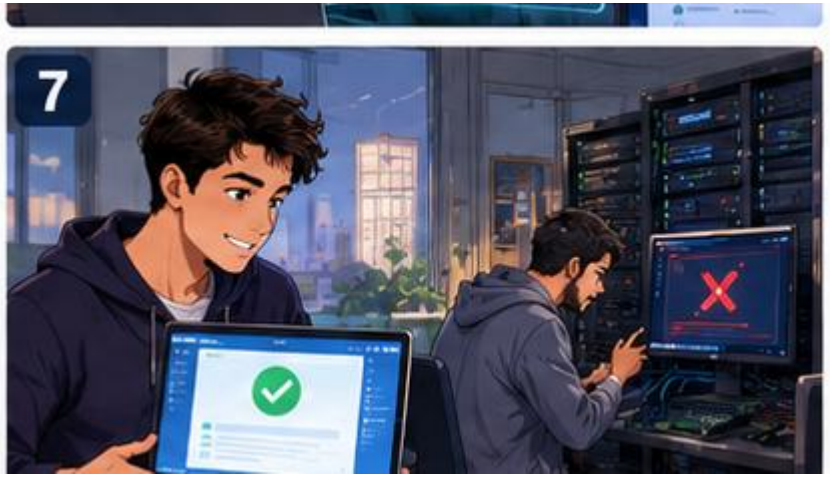
« Comment avez-vous vérifié la sécurité de votre code ? »

Les étudiants répondent :

« Nous allions le faire à la fin… »

Le responsable trouve :

- ❌ une mauvaise gestion des mots de passe
- ❌ une API insuffisamment protégée
- ❌ une validation des entrées insuffisante



Le déploiement est bloqué.

Question :

« Pourquoi attendre la fin du projet pour découvrir ces problèmes ? »

Motivation (8/10)

« Ça fonctionne chez moi ! »

Vous essayez maintenant de déployer l'application.

Développeur :

« Chez moi, ça fonctionne ! »



Serveur universitaire :

- ❌ configuration différente
- ❌ dépendance manquante
- ❌ erreur de configuration

Question :

« Comment rendre le déploiement plus fiable et reproductible ? »

Motivation (9/10)

Le vrai problème

Le projet est en difficulté

Vous avez pourtant des étudiants compétents.
Mais :

- **Communication ✕**
- **Intégration ✕**
- **Tests ✕**
- **Déploiement ✕**
- **Sécurité ✕**
- **Gestion des changements ✕**



Question centrale

« Le problème vient-il réellement du manque de compétences techniques ? »

« Ou vient-il de notre manière d'organiser le développement ? »

Motivation (10/10)

Votre deuxième mission

🔄 Recommencez le projet !

Cette fois :

Comment organiseriez-vous votre travail pour éviter les problèmes que vous venez de rencontrer ?

Les étudiants proposent progressivement :

Collaboration
↓
Intégration fréquente
↓
Tests automatisés
↓
Sécurité intégrée
↓
Déploiement automatisé
↓
Monitoring
↓
Feedback continu











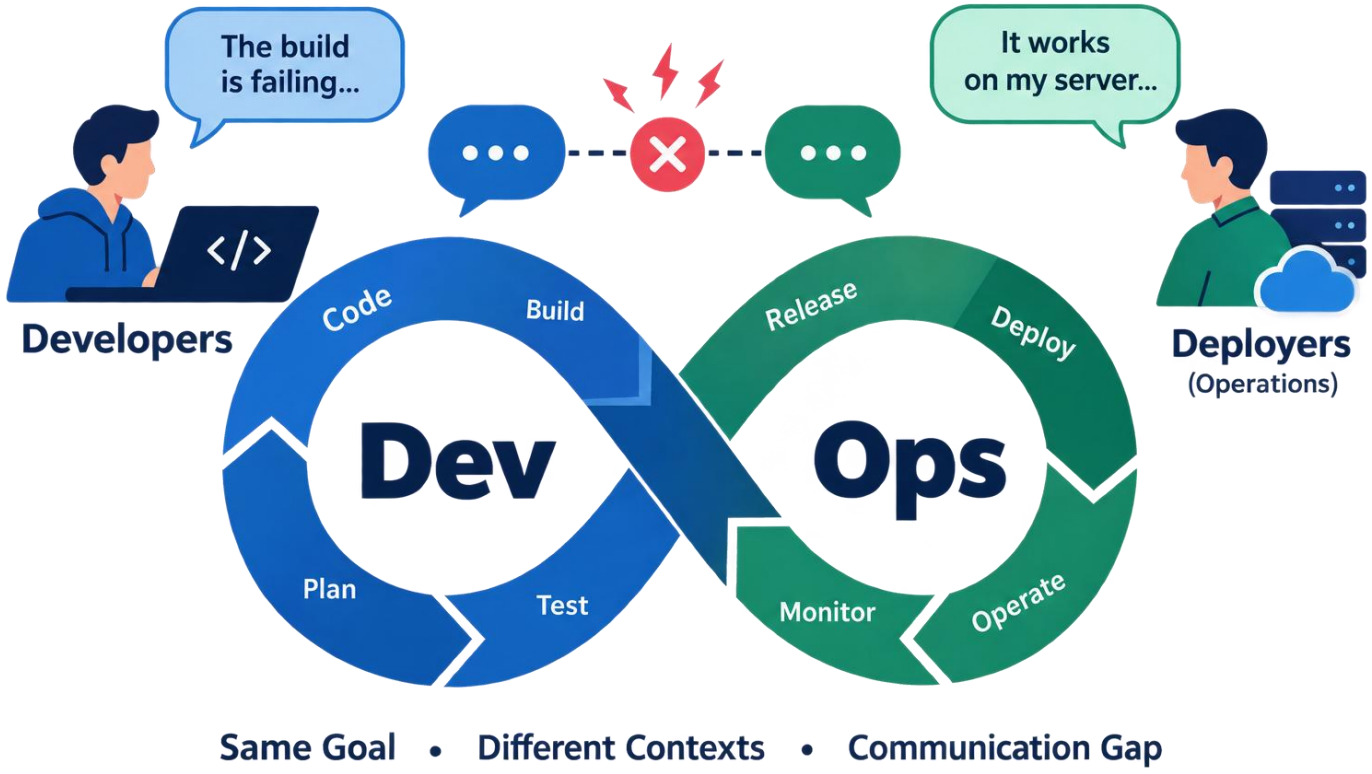
DevOps

Vous venez de construire DevOps (**Development + Operations**)



DevOps

Abdelkader Ouared



À propos du module DevOps

- **Intitulé du master : Sécurité informatique**
- **Semestre :** S1
  - **UE :** UET1 : première unité d'enseignement transversale.
  - **Titre de la matière : DevOps**
  - **Crédit :** 2
  - **Coefficient :** 1
- **Objectifs de la matière**

- les étudiants seront capables de gérer des projets logiciels avec Agile et DevOps, d'automatiser les pipelines CI/CD, d'utiliser l'IaC et la conteneurisation, et d'intégrer la sécurité (DevSecOps).

- **Connaissances préalables recommandées**

- Principes de base du développement logiciel (programmation et contrôle des versions), fondamentaux des réseaux informatiques et de la cybersécurité, et familiarité avec les concepts du Cloud Computing.

À propos du module DevOps

- **Contenu de la matière**
  1. Introduction à la gestion de projets logiciels en DevOps
  2. Méthodes de développement agiles en DevOps
  3. Contrôle de version et collaboration
  4. Intégration et déploiement continus (Continuous Integration & Continuous Deployment : CI/CD)
  5. Infrastructure as Code (IaC) et conteneurisation
  6. DevOps basé sur le Cloud
  7. Surveillance, journalisation et optimisation des performances
  8. DevSecOps : intégration de la sécurité dans DevOps
- **Mode d'évaluation**
  - **Contrôle continu et examen.**

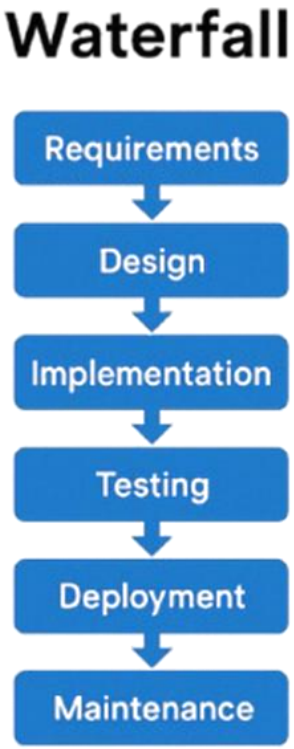
Les techniques/méthodes pédagogiques utilisées

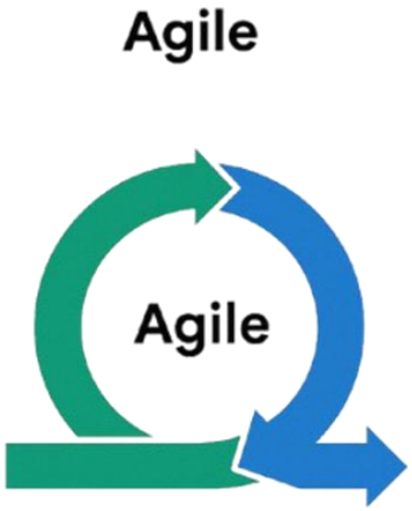
Technique	Définition	Méthode
Cours magistral **(Lecture)**	Il s'agit de la méthode d'enseignement traditionnelle dans laquelle l'enseignant transmet directement les connaissances aux étudiants.	Transmission
Travaux pratiques en laboratoire **(Labs)**	Ils consistent à réaliser des tâches pratiques permettant d'explorer un sujet d'ingénierie logicielle, généralement dans des salles dédiées équipées d'un ordinateur par étudiant.	Transmission
Processus agile **(Agile Process)**	Il consiste à utiliser des activités agiles pendant le déroulement du projet, telles que les sprints et la planification Scrum.	Gestion de projet
Apprentissage par projet (**Project-Based Learning**)	Il est centré sur un projet dans lequel les étudiants travaillent sur une tâche concrète.	Active

Cours 1

- **Introduction à la gestion de projets logiciels en DevOps**
  - DevOps dans le contexte de l'ingénierie logicielle et de la sécurité des réseaux.
  - Différences entre la gestion de projet traditionnelle et Agile et DevOps.
  - Collaboration entre les équipes de développement, d'exploitation et de sécurité.

Waterfall vs Agile vs DevOps





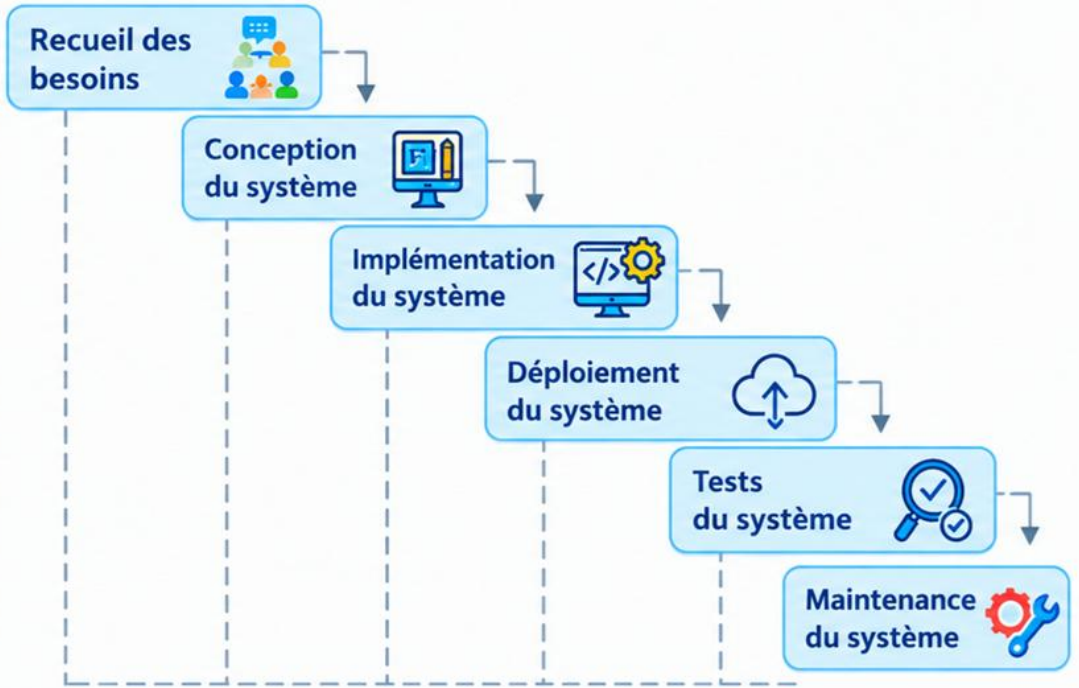


Le modèle en cascade (The waterfall model)

Le modèle en cascade **est un processus où le développement commence seulement après la complétion de la phase précédente.**

Avantages

- Simple à comprendre et à utiliser.
- Facile à gérer grâce à des phases bien définies.
- Livrables et jalons clairement identifiés.
- Adapté aux petits projets aux exigences stables.
- Tâches faciles à planifier.
- Documentation claire et structurée

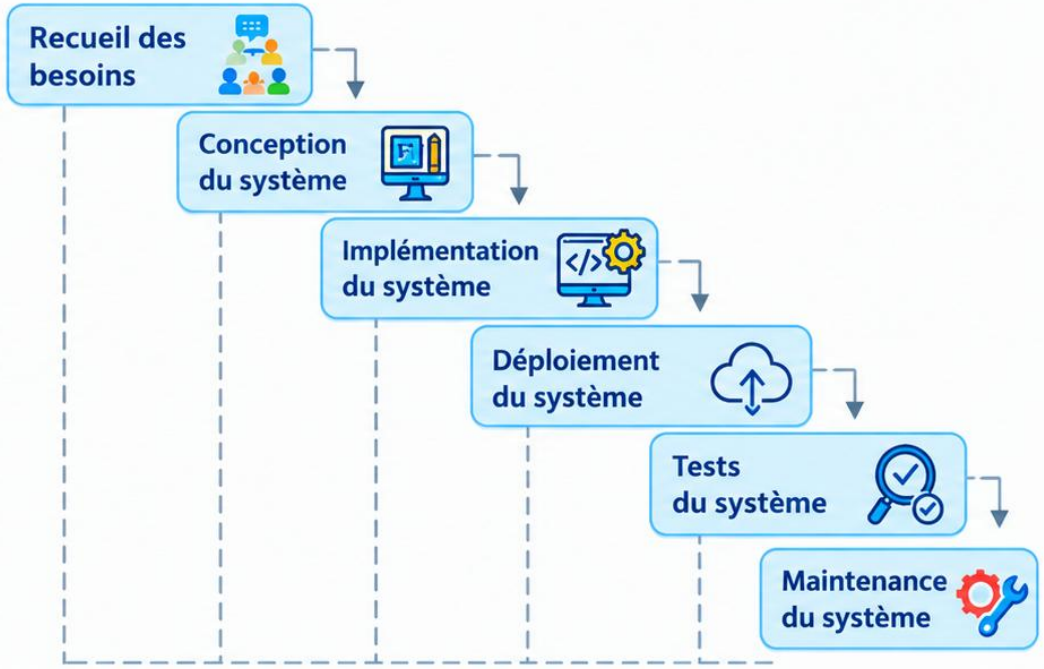


Le modèle en cascade (The waterfall model)

Le modèle en cascade **est un processus où le développement commence seulement après la complétion de la phase précédente.**

Difficulté dans cette approche

- La phase de ***développement*** dure ***plus longtemps***.
- Le produit est livré au client sur le marché uniquement après la ***modification de la demande***.
- En conséquence, le client peut perdre de l'intérêt pour le produit en raison du ***développement prolongé*** ou d'un ***changement de ses besoins***.



Le Manifeste Agile et les méthodologies agiles

- **Le Manifeste Agile (2001)**
  - Créé par 17 experts, aux États-Unis.
  - Réaction aux limites des méthodes traditionnelles.
  - **Valeur clé :** privilégier les individus et leurs interactions.
  - Définit des valeurs et des principes pour un développement logiciel plus flexible et collaboratif.

Le Manifeste Agile et les méthodologies agiles

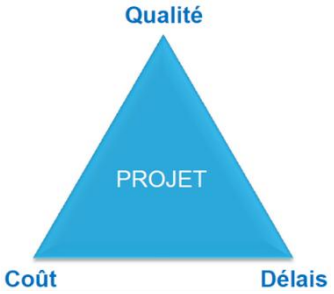
Qu’est-ce que la réussite ?



❑ Management Classique

Taux de succès des projets Digitaux et IT (Standish Group**)

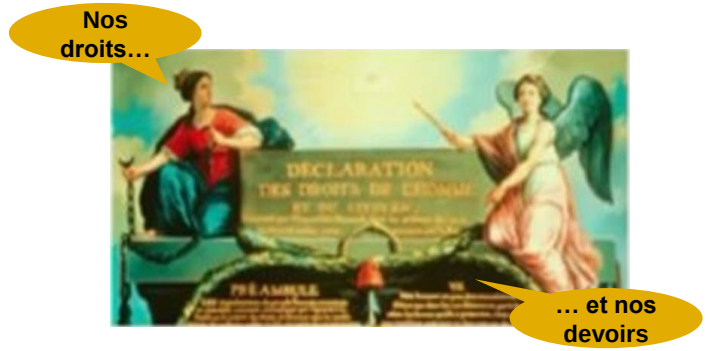
Projets réussis
(31%)



❑ Management Agile (~2001)



La Qualité des relations



Créer un cadre sécurisée (3P)

Le Manifeste Agile et les méthodologies agiles

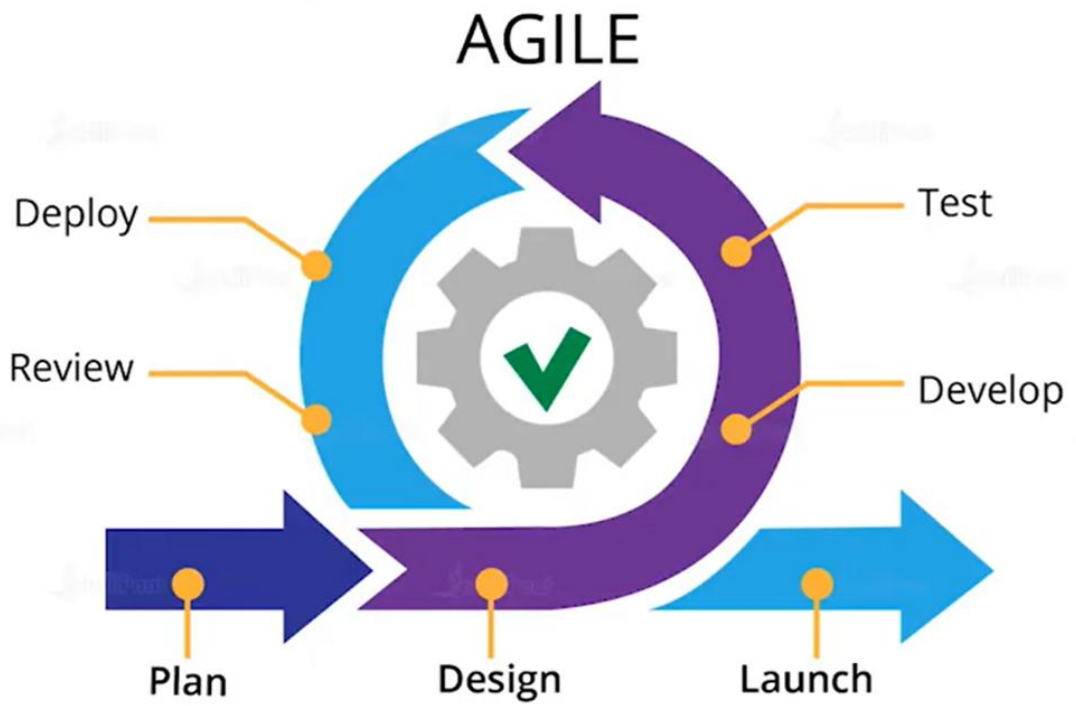
Le projet Aristote: Équipes performantes **vs** Équipes dysfonctionnelles





https://viuz.com/2016/03/04/projet-aristote-les-cinq-cles-des-equipes-gagnantes-selon-google/

Qu’est-ce que la méthodologie Agile ?



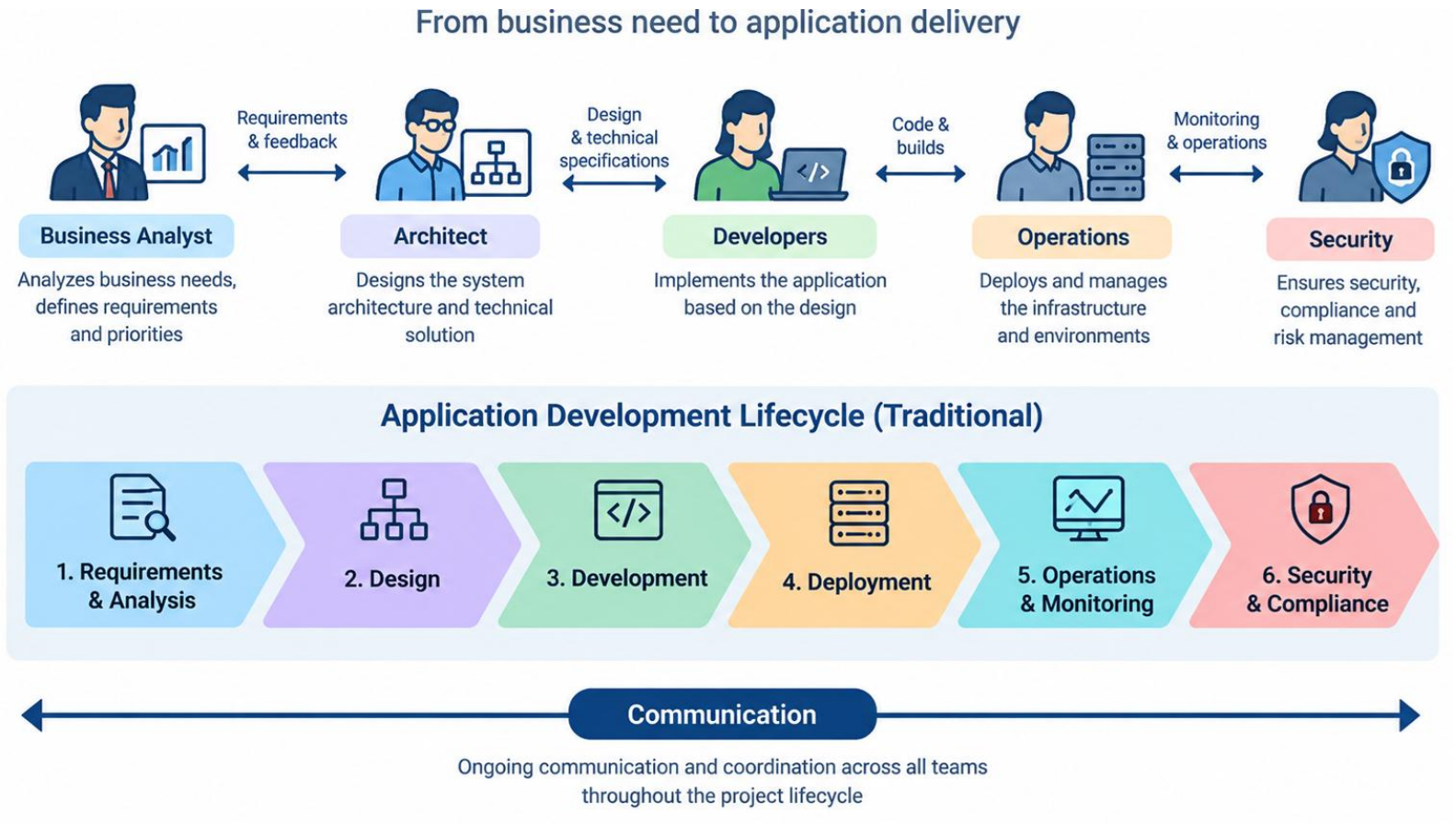
- The **Waterfall model** is **sequential**, whereas the **Agile model** is **incremental** and **iterative.**

Qu’est-ce que la méthodologie Agile ?

Comparaison entre les méthodologies Waterfall et Agile

Waterfall (Cascade)	Agile
Séquentielle	Flexible
Plan fixe	Adaptative
Tests à la fin	Tests continus
Implication limitée du client	Implication continue du client
Documentation détaillée	Documentation allégée
Risques anticipés en amont	Gestion continue des risques

Traditional Software Development (Before DevOps)



Le modèle DevOps

Qu'est-ce que le DevOps ?

Le DevOps est une stratégie qui fait le lien entre les équipes de **développement** (Development) et d'**exploitation** (Operations) afin de livrer des logiciels de **bonne qualité dans les délais prévus.**

Objectifs du DevOps

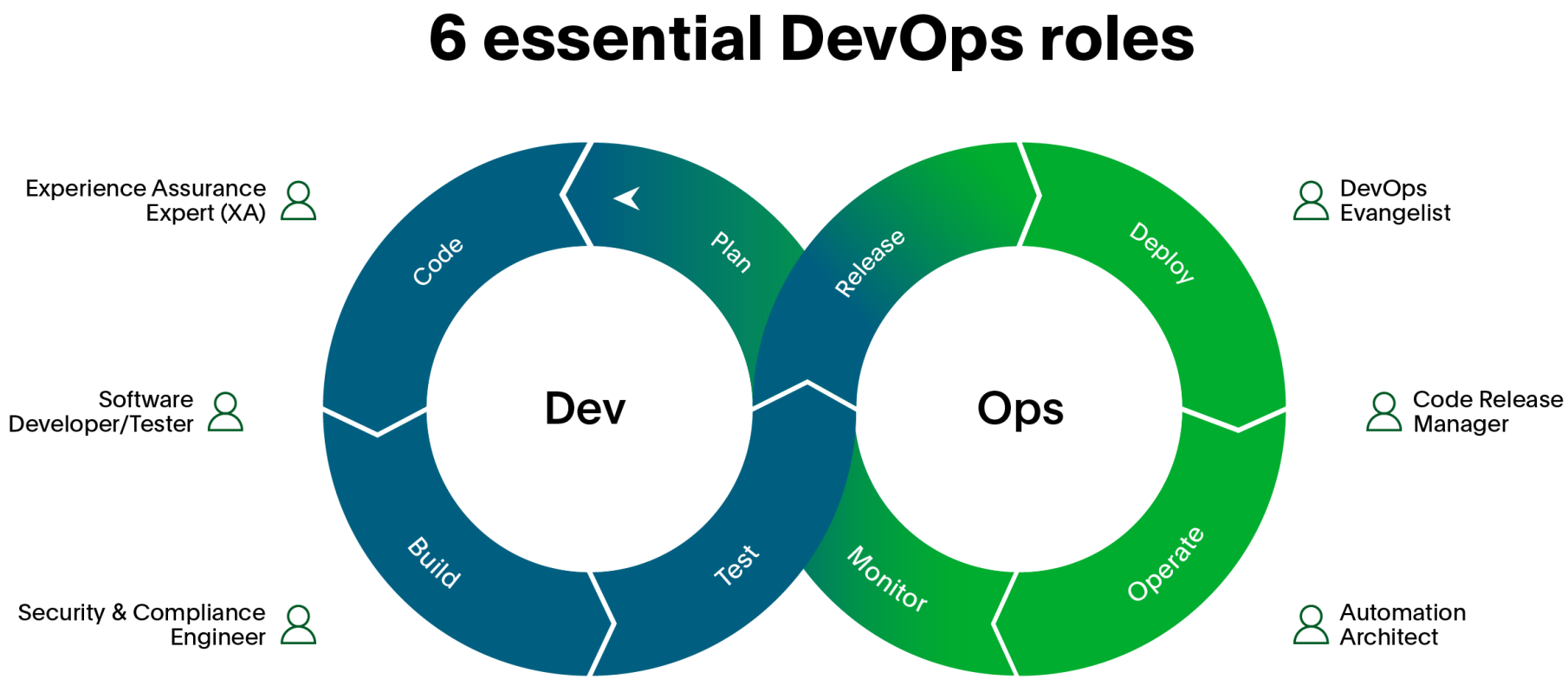
- **Accélérer la livraison :** transférer rapidement, facilement, de manière répétée et en toute sécurité les logiciels de l'environnement de développement vers l'environnement de production.
- **Réduire les limites des méthodes traditionnelles :** améliorer les processus de développement et de livraison en favorisant la collaboration et l'automatisation.
- **Améliorer la collaboration et la productivité** grâce à :
  - L'automatisation de l'infrastructure.
  - L'automatisation des flux de travail (workflows).
  - La surveillance continue des performances des applications.



Dev + Ops = DevOps

Une collaboration continue, soutenue par l'automatisation, pour livrer des logiciels fiables plus rapidement.

Le modèle DevOps



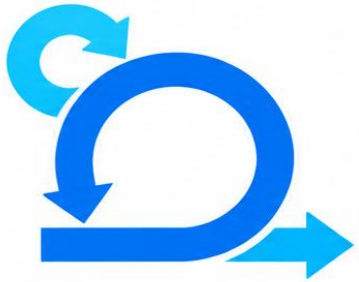
DevOps vs Agile

DevOps



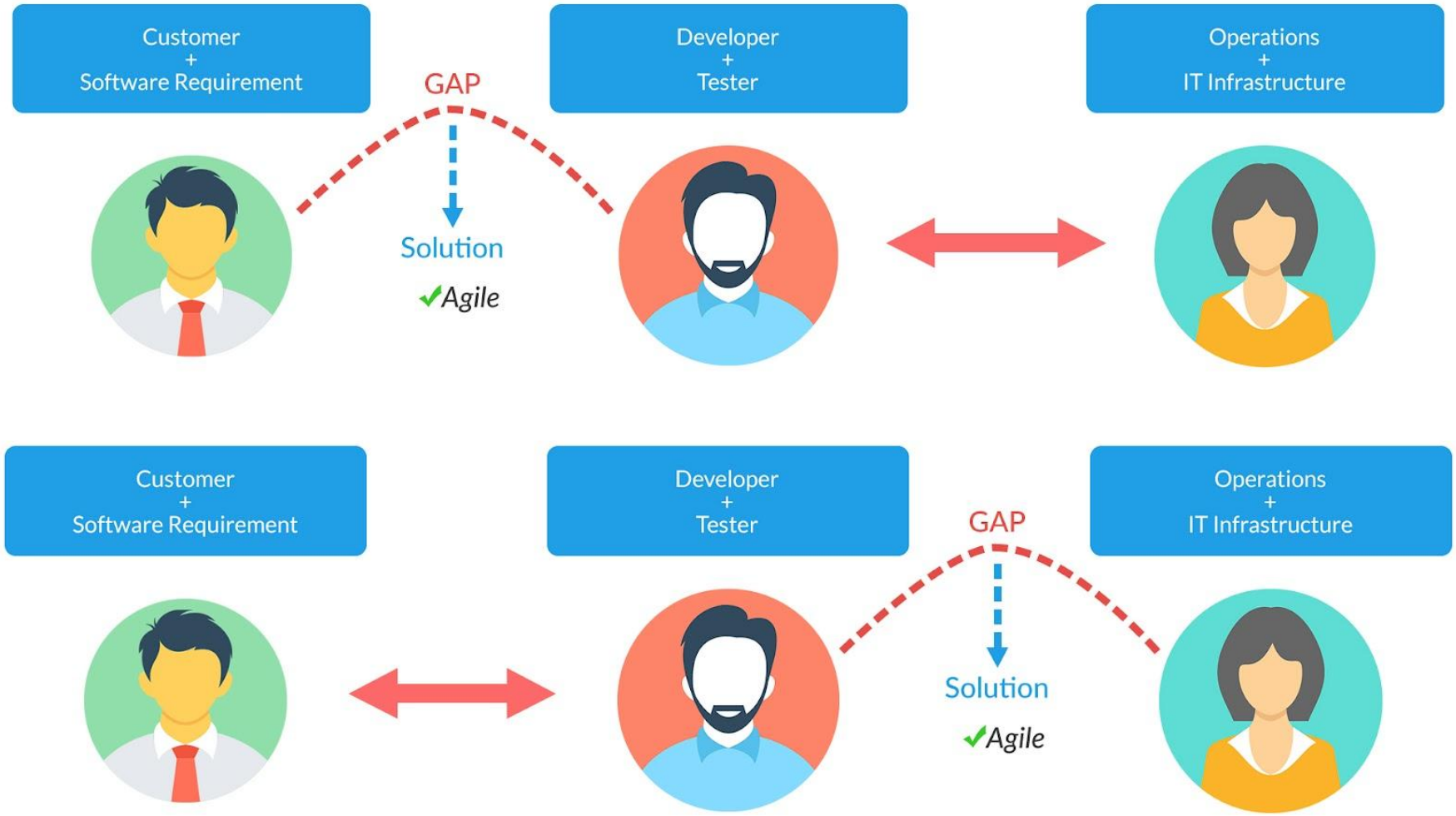
Le DevOps est une culture qui met l'accent sur la collaboration entre les équipes de développement et d'exploitation, ainsi que sur l'automatisation de l'ensemble du processus.

Agile

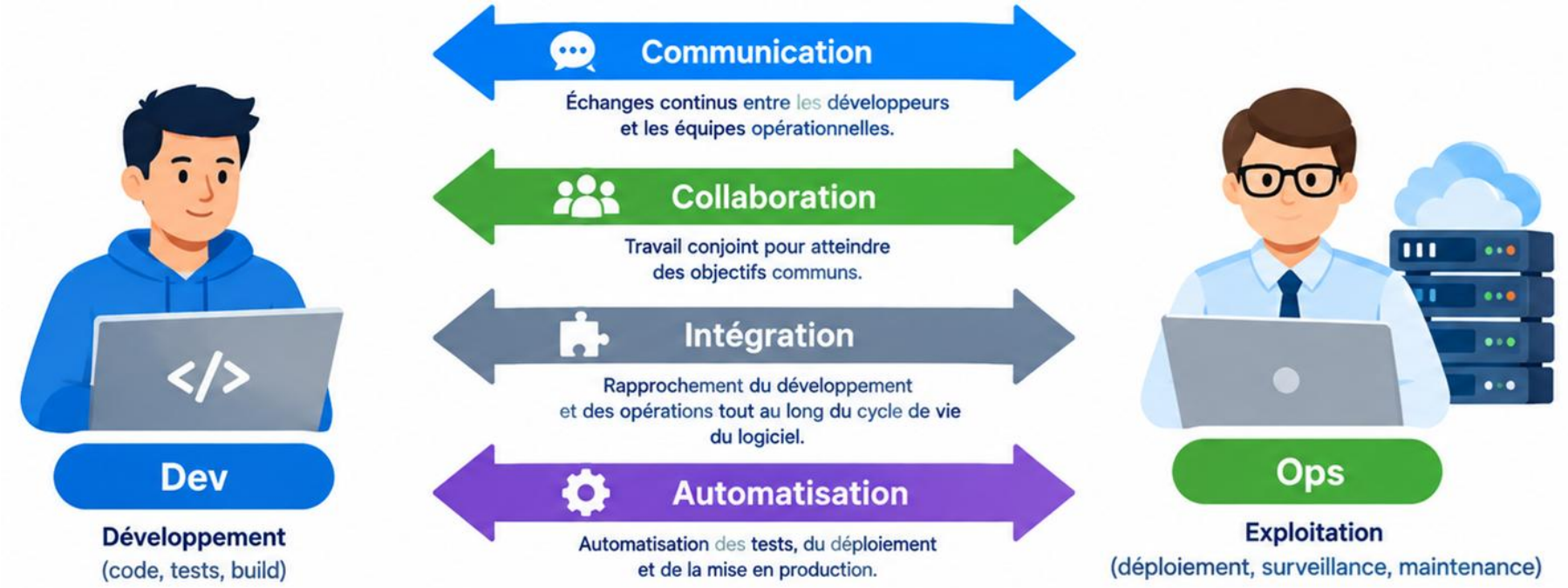


L'Agile est une approche qui privilégie la collaboration avec le client et l'adaptation au changement plutôt que le suivi strict d'un plan.

DevOps vs Agile



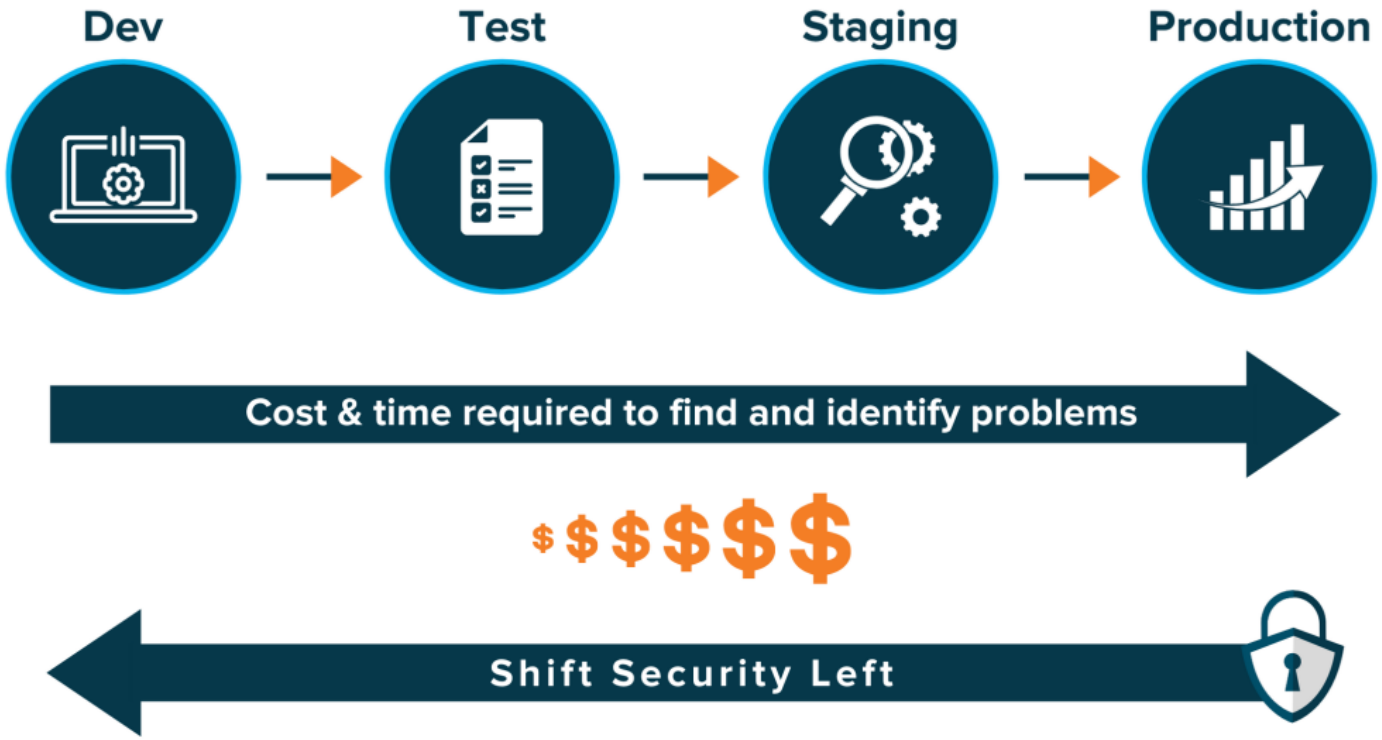
DevOps vs Agile



DevOps : Développement Agile + Opérations IT

L'Agile améliore la manière de développer les logiciels,
tandis que DevOps étend cette approche à leur déploiement, leur exploitation et leur maintenance.

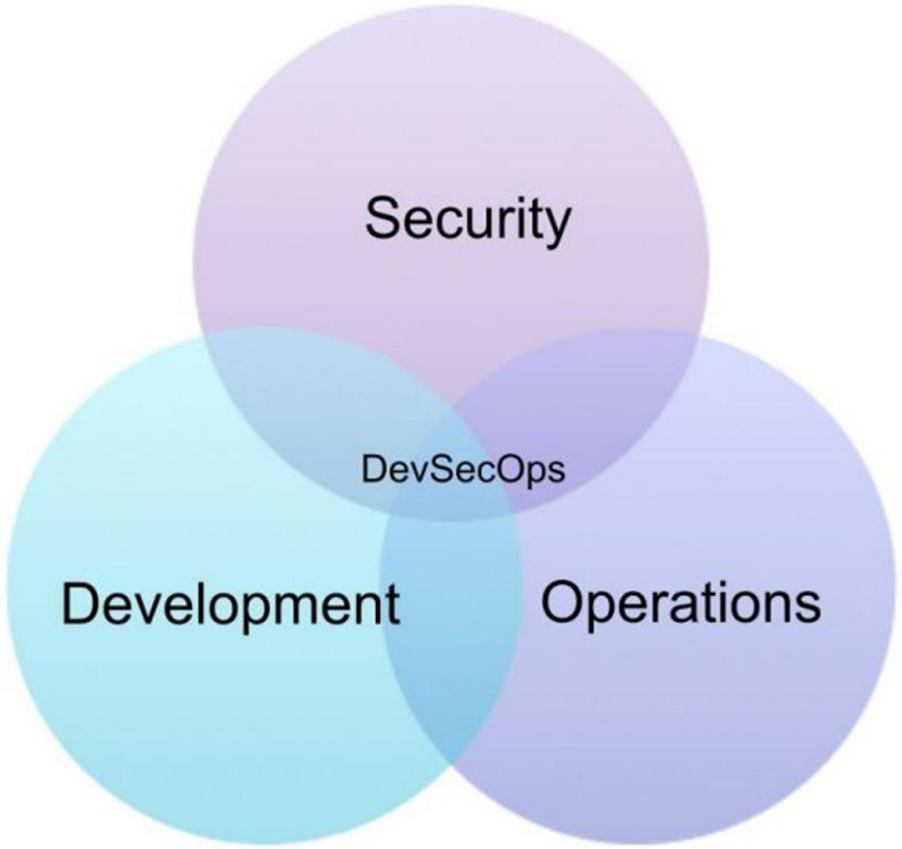
Qu’est-ce que la sécurité décalée vers la gauche (Shift Left Security) ?



DevSecOps

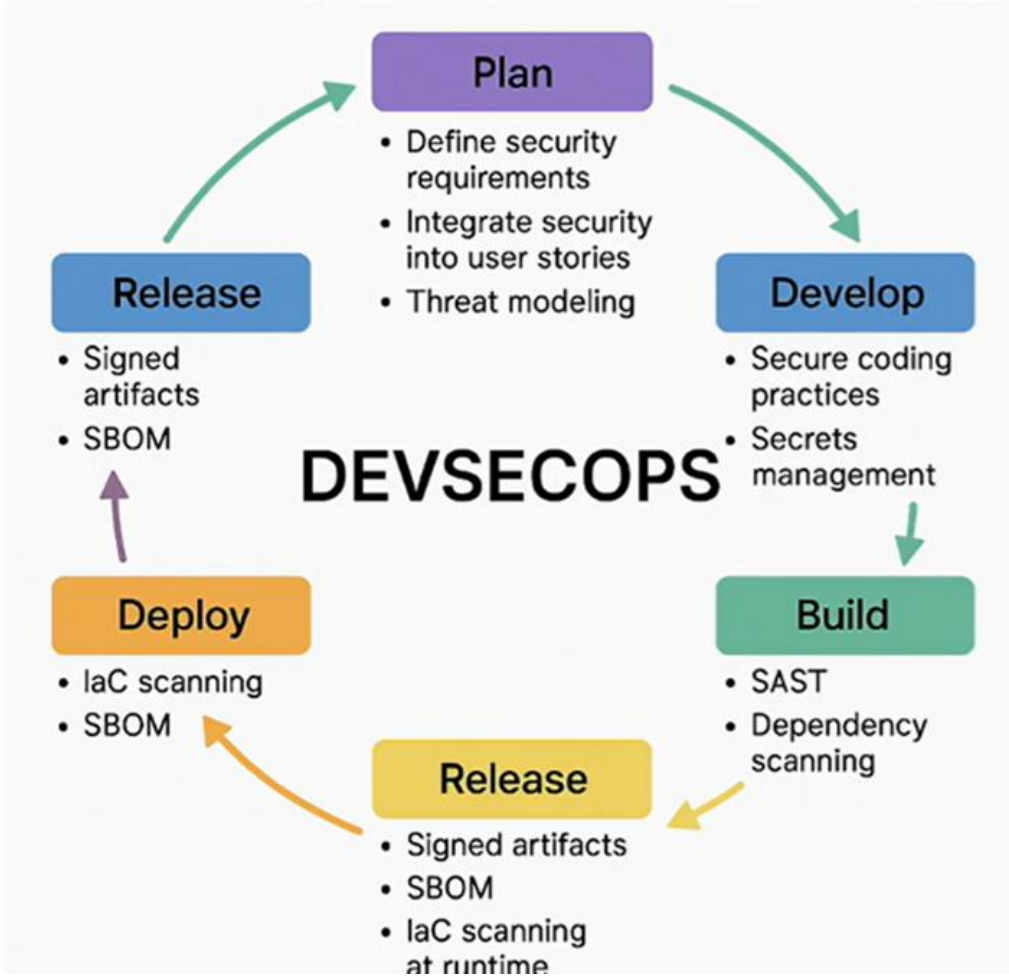
Qu’est-ce que le DevSecOps ?

- **DevSecOps =** Développement + Sécurité + Exploitation
- Intègre les pratiques de sécurité tout au long du processus DevOps.
- **Objectif :** sécuriser la chaîne de livraison logicielle sans ralentir le développement.



DevSecOps Pipline

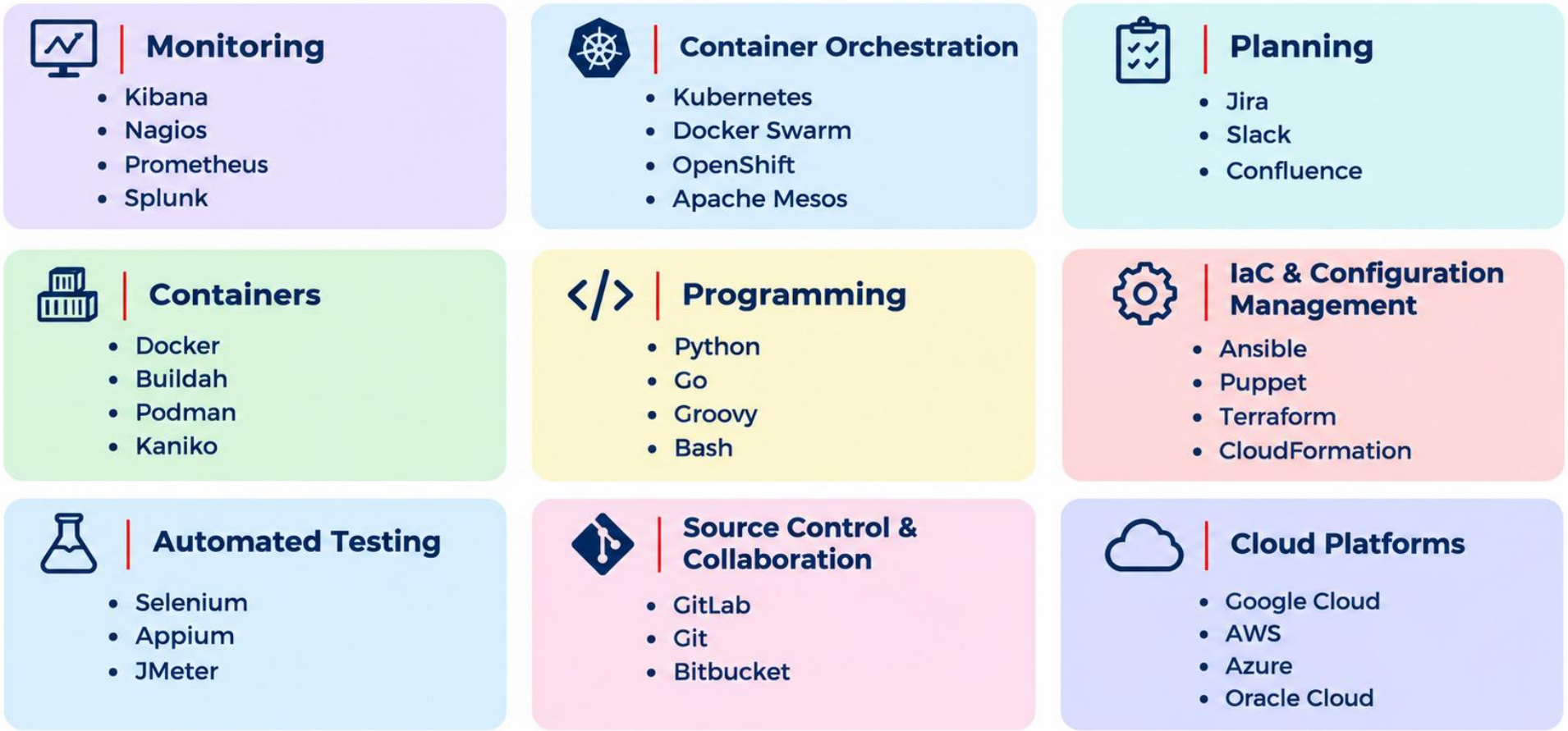
Instaurer une culture DevSecOps pour intégrer la sécurité dès les premières étapes du cycle de développement (*Shift Left*) et tout au long du pipeline **DevSecOps**.



DevOps vs DevSecOps



Les outils DevOps



Outil votre TP N 1

Objectif pédagogique	Outil
Découvrir Agile et Kanban	Trello
Pratiquer Scrum et les Sprints	Jira
Développer une application en équipe	GitHub Projects







TP N 1: Mise en pratique de la méthode Agile avec Trello et Jira : simulation d’un projet client



Q&A



